COMP 511

Computer Programming Foundations II

Chapter 7 : Recursion

What is the Recursion?

A *recursive method* is a method that either directly or indirectly makes a call to itself.

Recursion is a powerful problem-solving tool.

Recursion is NOT a substitute for a simple loop.

How it works

- Recursion can always be removed by using a stack
- Too much recursion can be dangerous

Fundamental Rules of Recursion

1. *Base case:* always have at least one case that can be solved without using recursion.

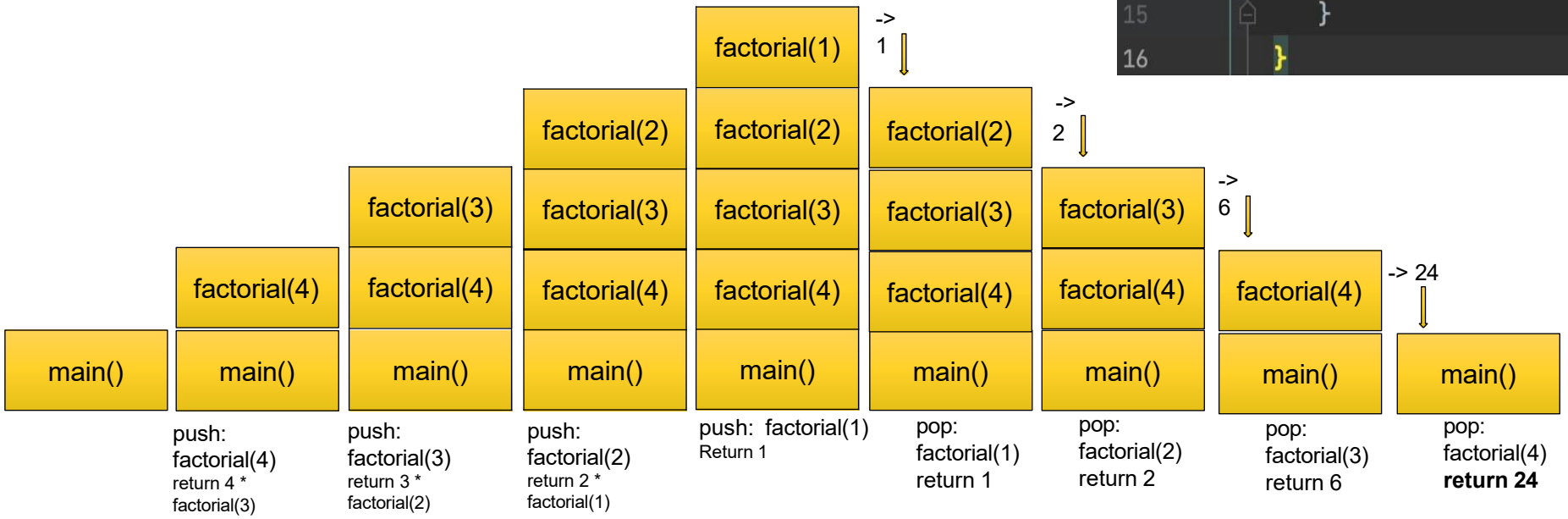
2. *Make progress:* Any recursive call must progress toward a base case.

```
// Evaluate the sum of the first n integers
public static long s( int n )
{
    if( n == 1 )
        return 1;
    else
        return s( n - 1 ) + n;
}
```

Recursion Example

Stack

```java
public class Recursion {

    public static int factorial(int a)
    {
        if (a == 1)   //base case
            return 1;
        else            // recursive step
            return a * factorial(a-1);
    }

    public static void main (String args[])
    {
        int i = 4;
        System.out.println ( i + "! = " + factorial(i));
    }
}
```



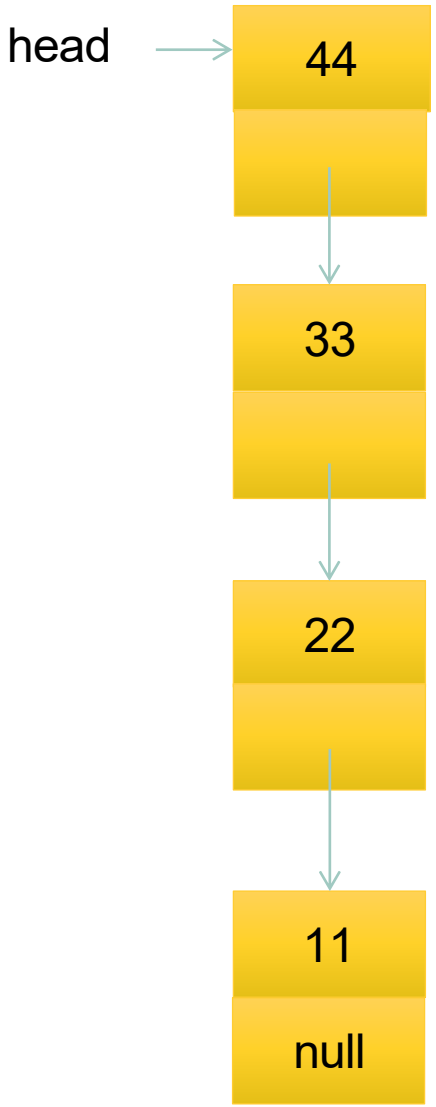
Recursion Exercise

```java
import java.util.*;
// For Class Demo only ...
public class recursion2 {
    static Node what(Node first) {
        if (first == null || first.next == null) {
            return first;
        }
        Node second = first.next;
        Node rest = what(second);
        second.next = first;
        first.next  = null;
        return rest;
    }

    public static void main(String[] args)  {
        LinkedList llist = new LinkedList();

        llist.insertNode(11);
        llist.insertNode(22);
        llist.insertNode(33);
        llist.insertNode(44);

        printLList(llist.head, " ");

        System.out.println();

        llist.head = what(llist.head);
        printLList(new_llist, " ");
    }
}
```

Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

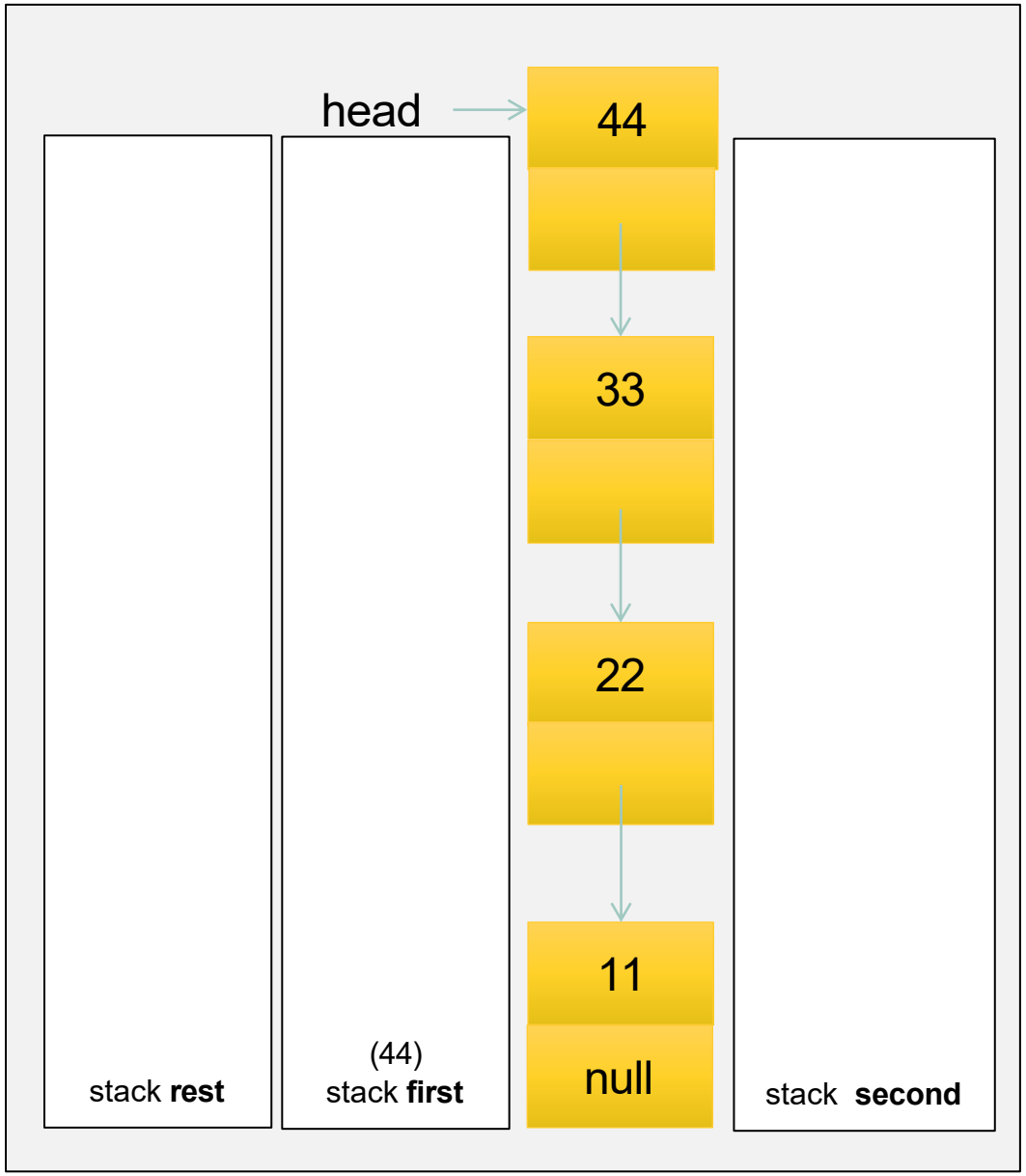
Tracker
Line 23



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

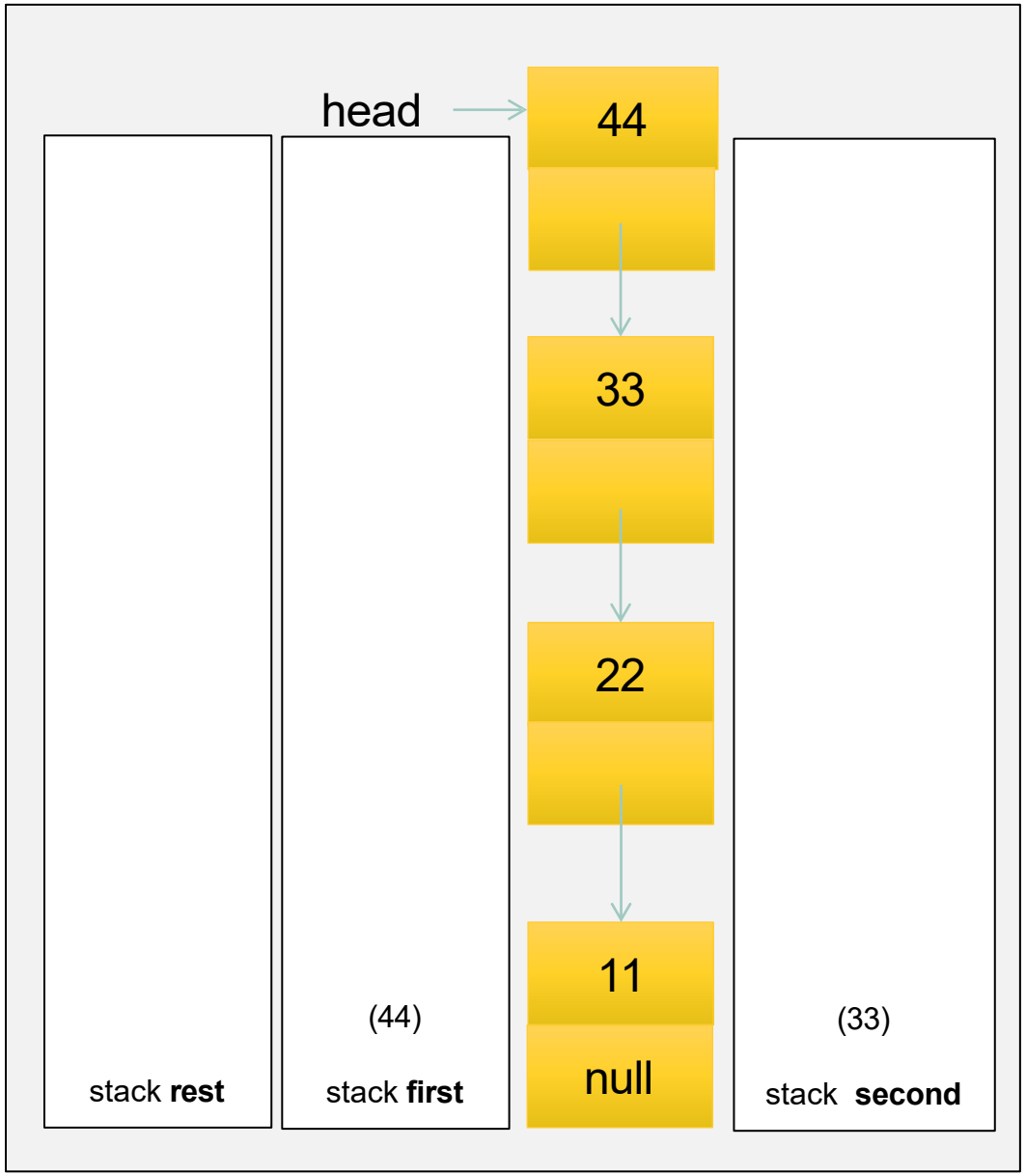
Tracker
Line 5



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

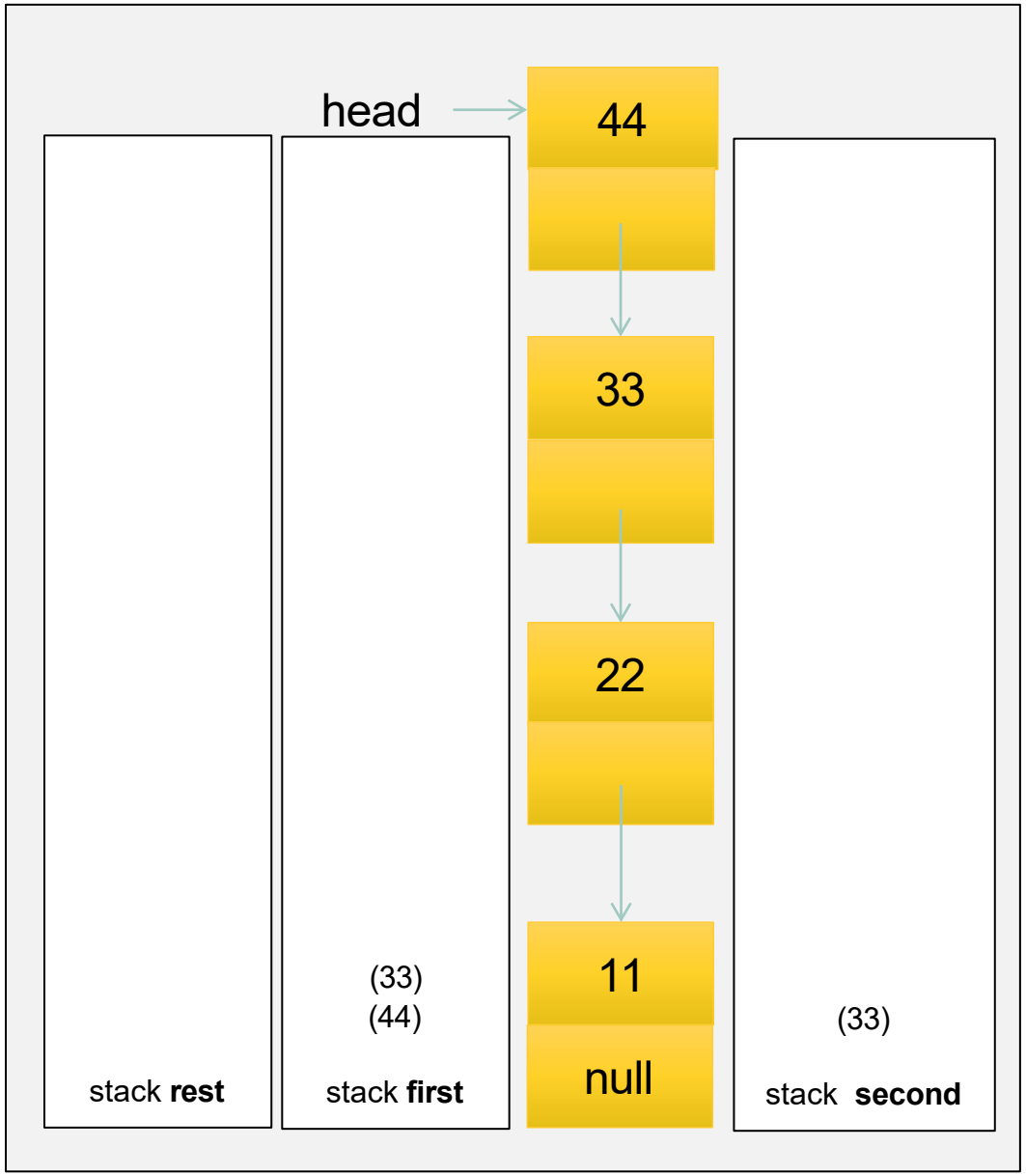
Tracker
Line 8



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

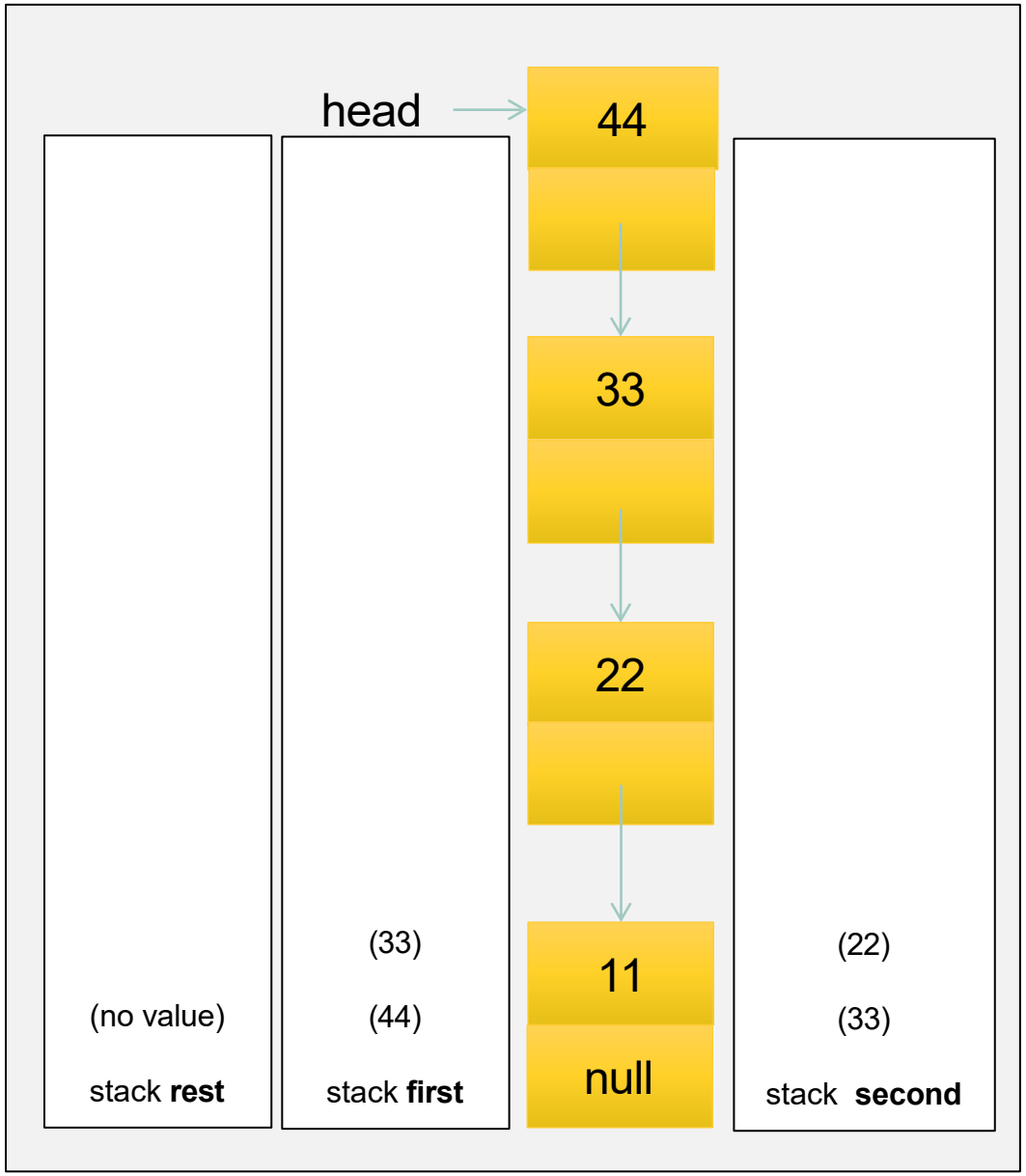
Tracker
Line 9



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

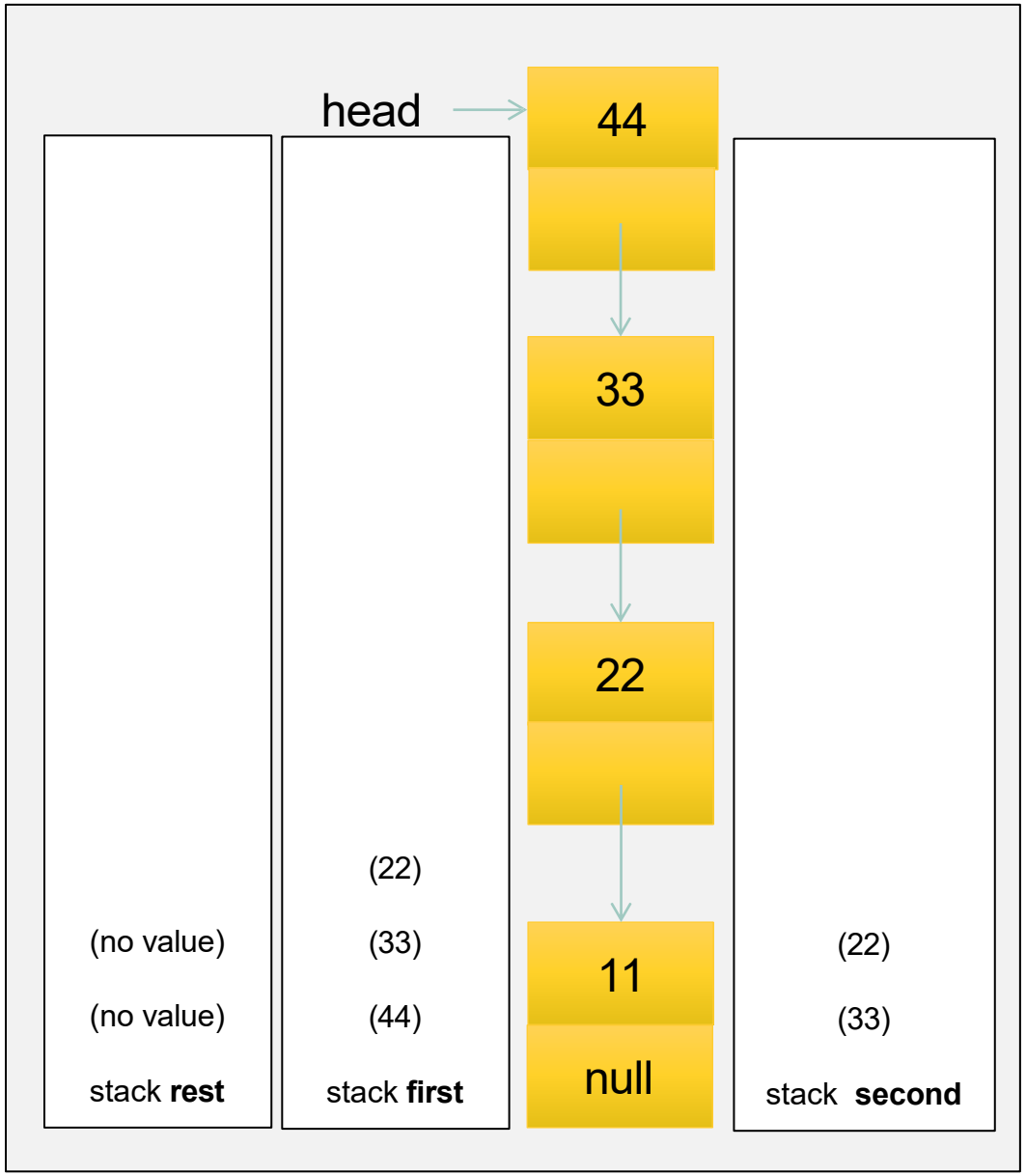
Tracker
Line 8



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

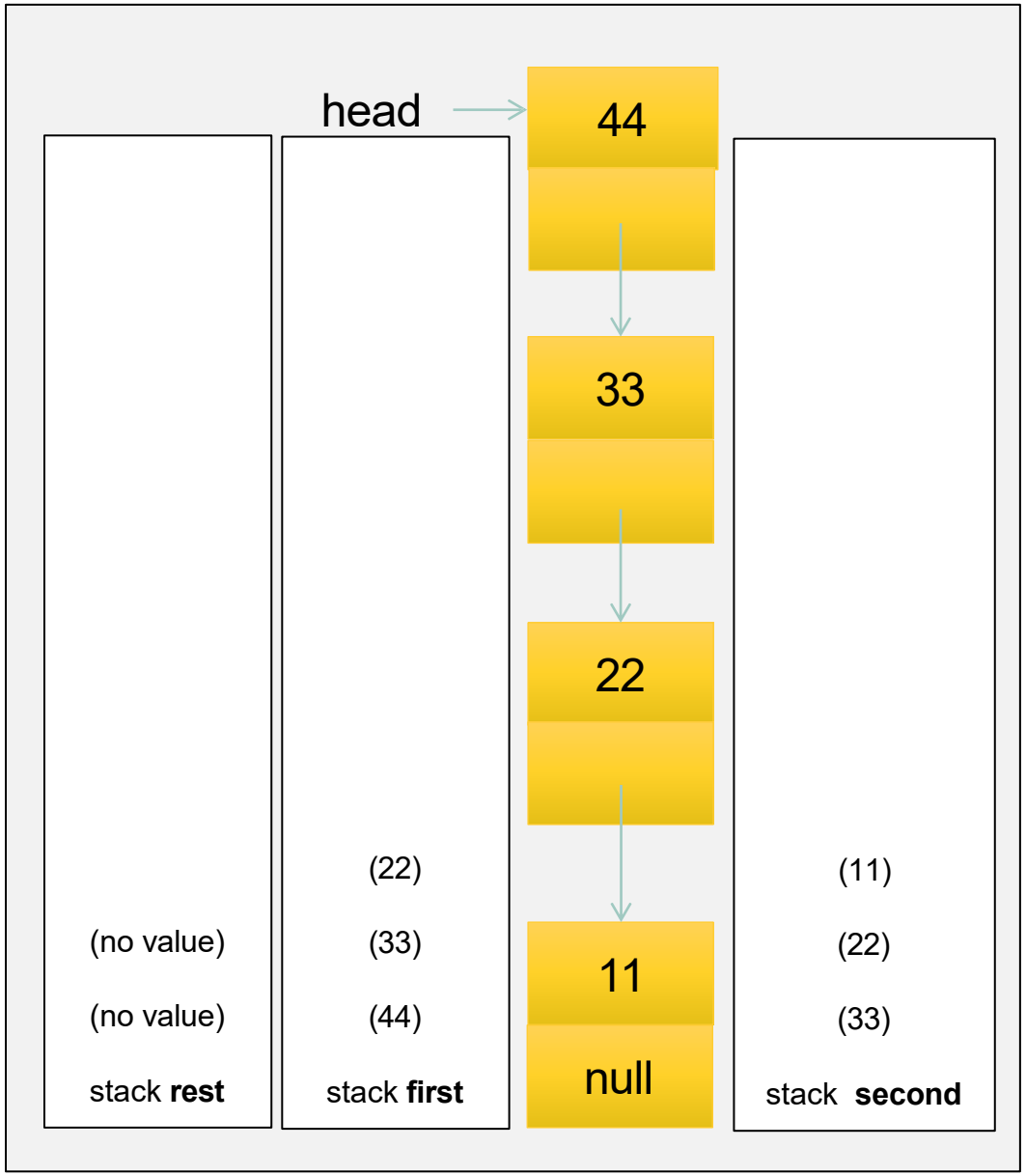
Tracker
Line 9



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

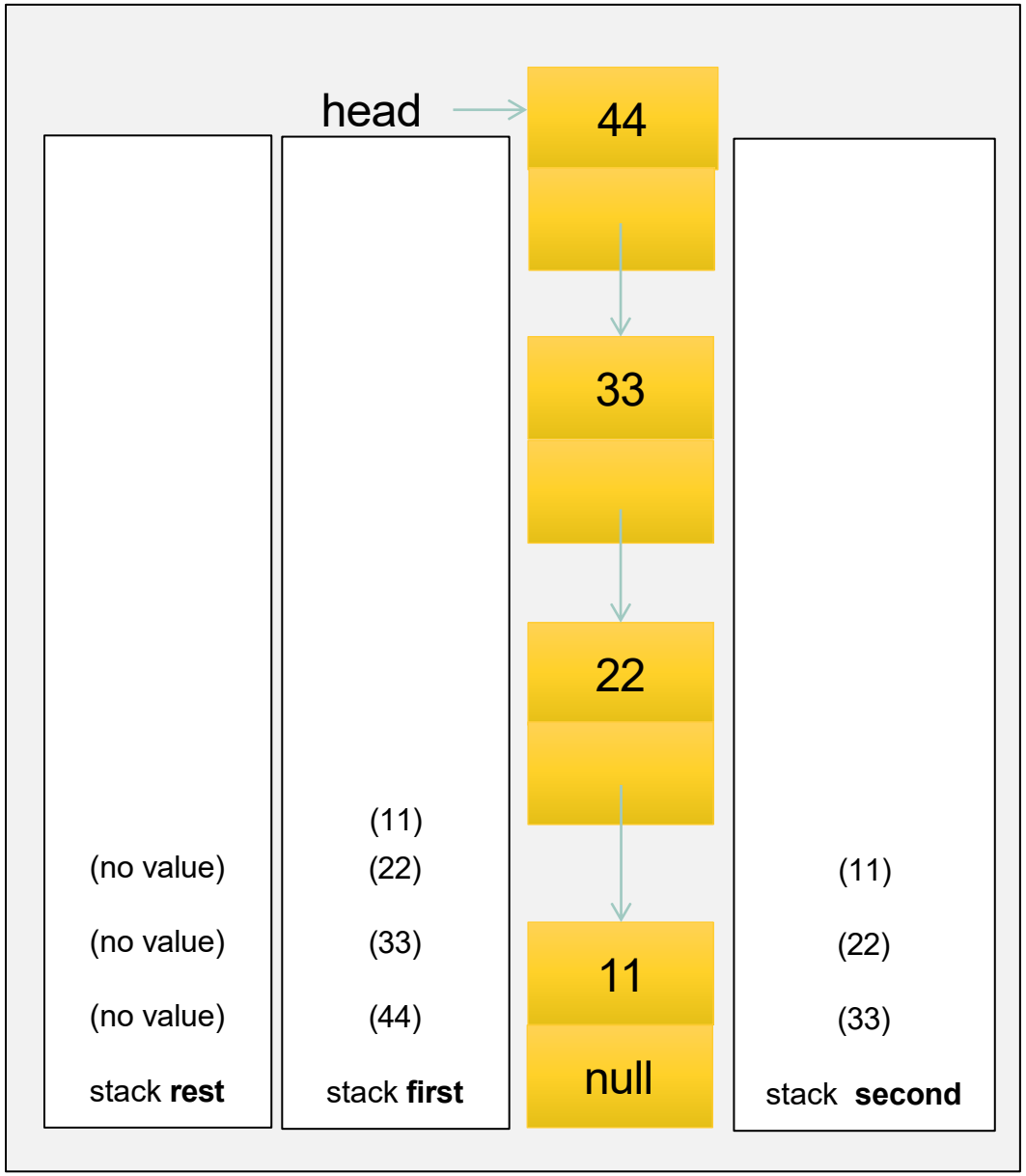
Tracker
Line 8



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

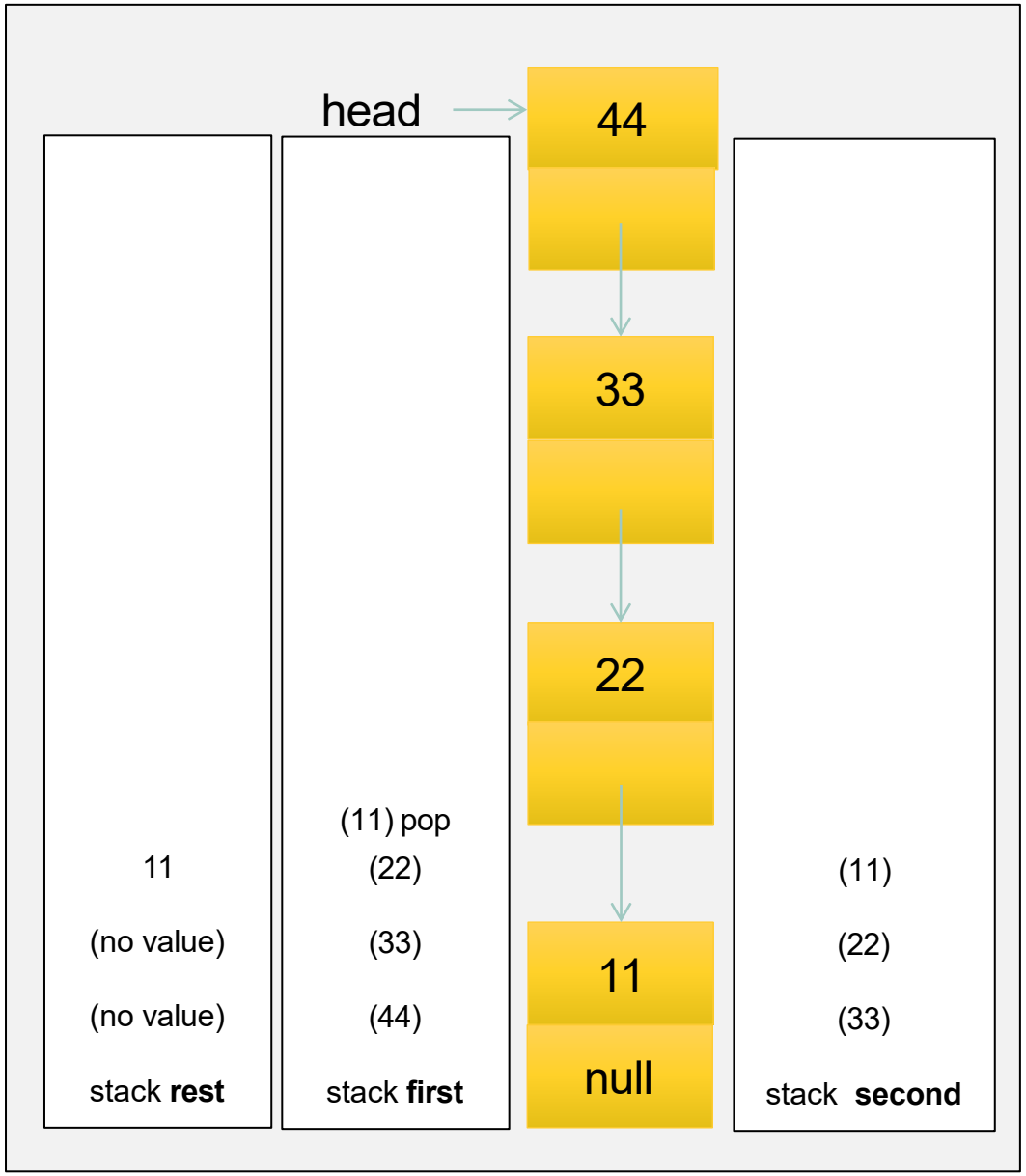
Tracker
Line 9



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

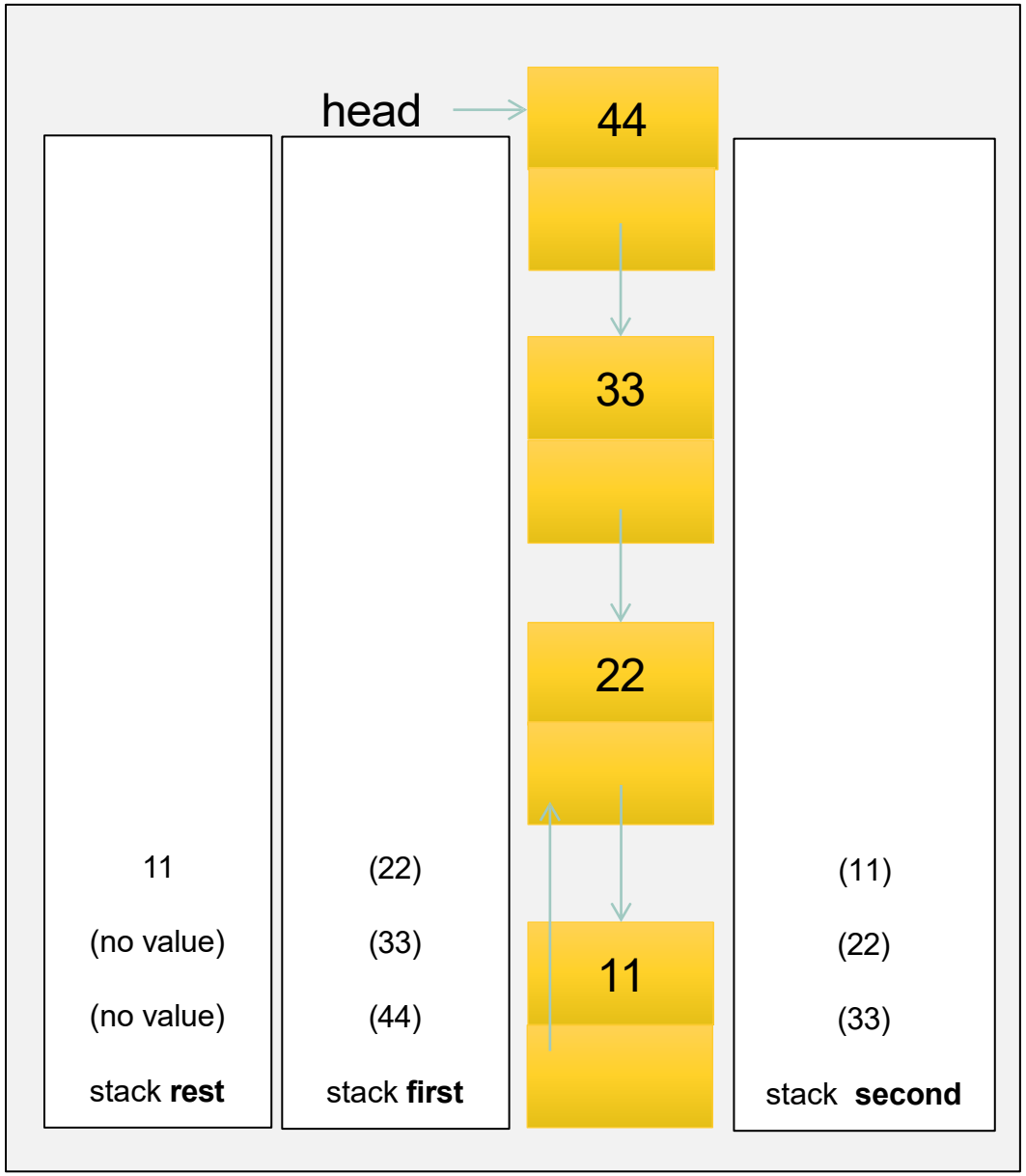
Tracker
Line 5



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

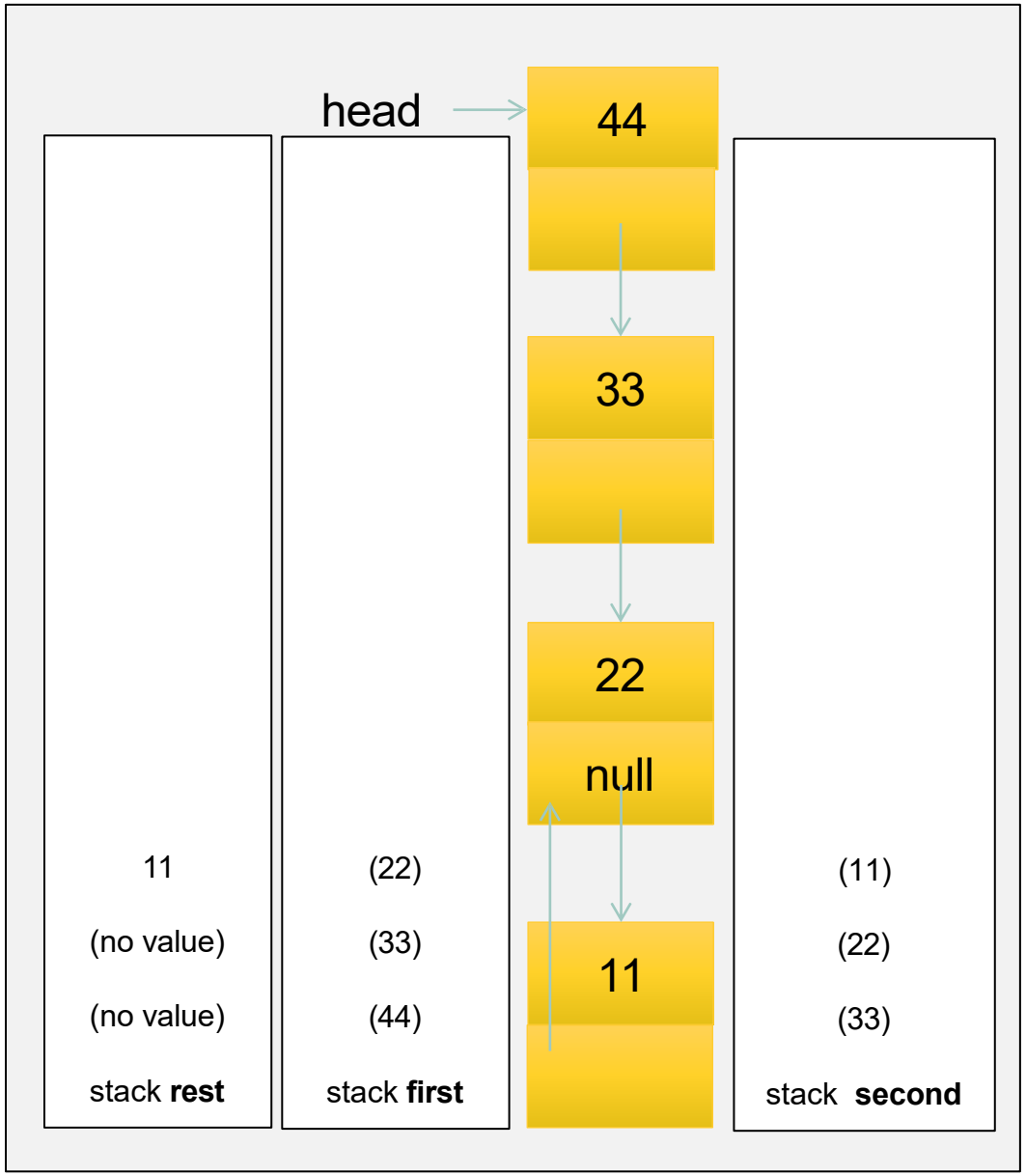
Tracker
Line 10



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

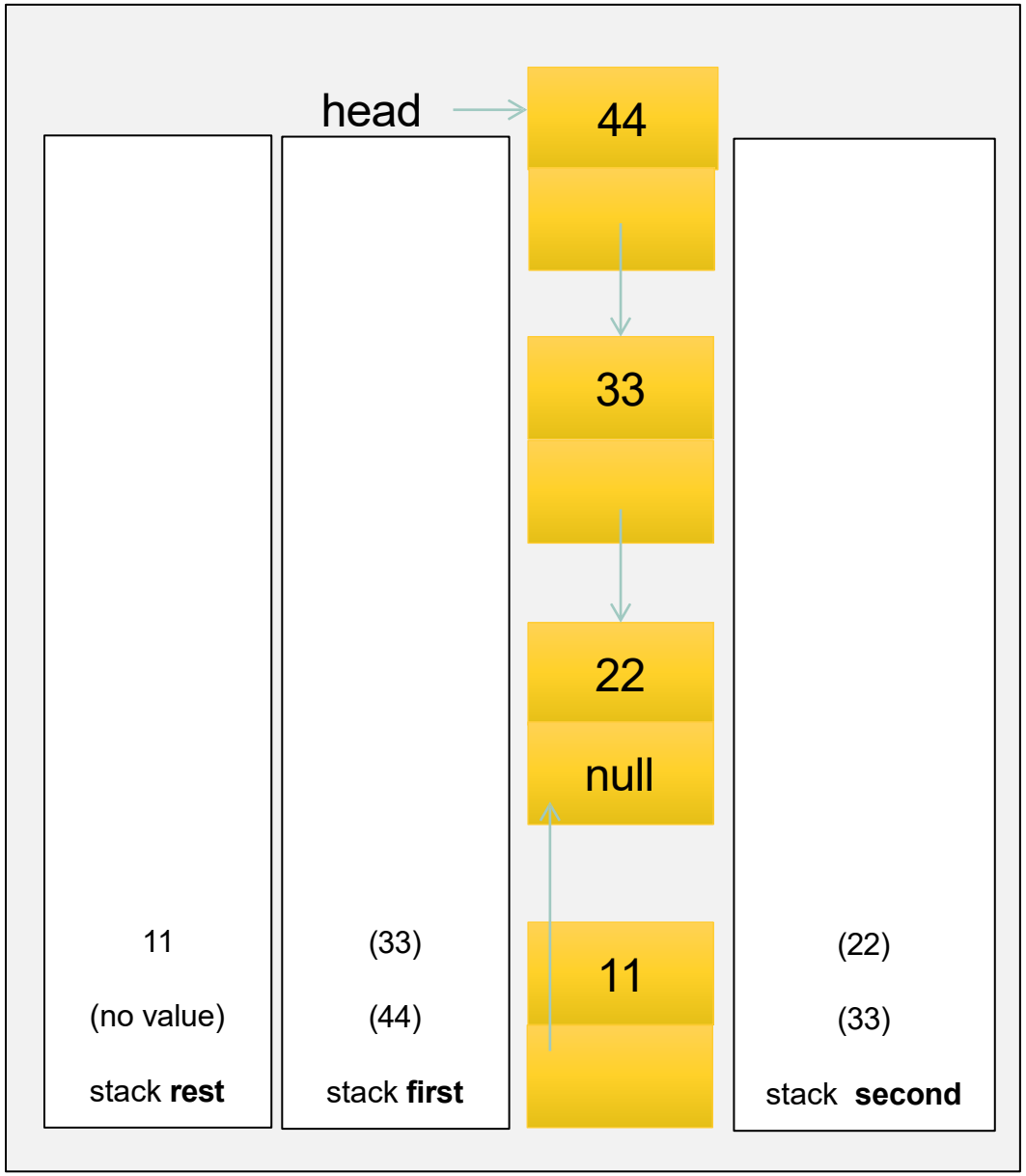
Tracker
Line 11



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

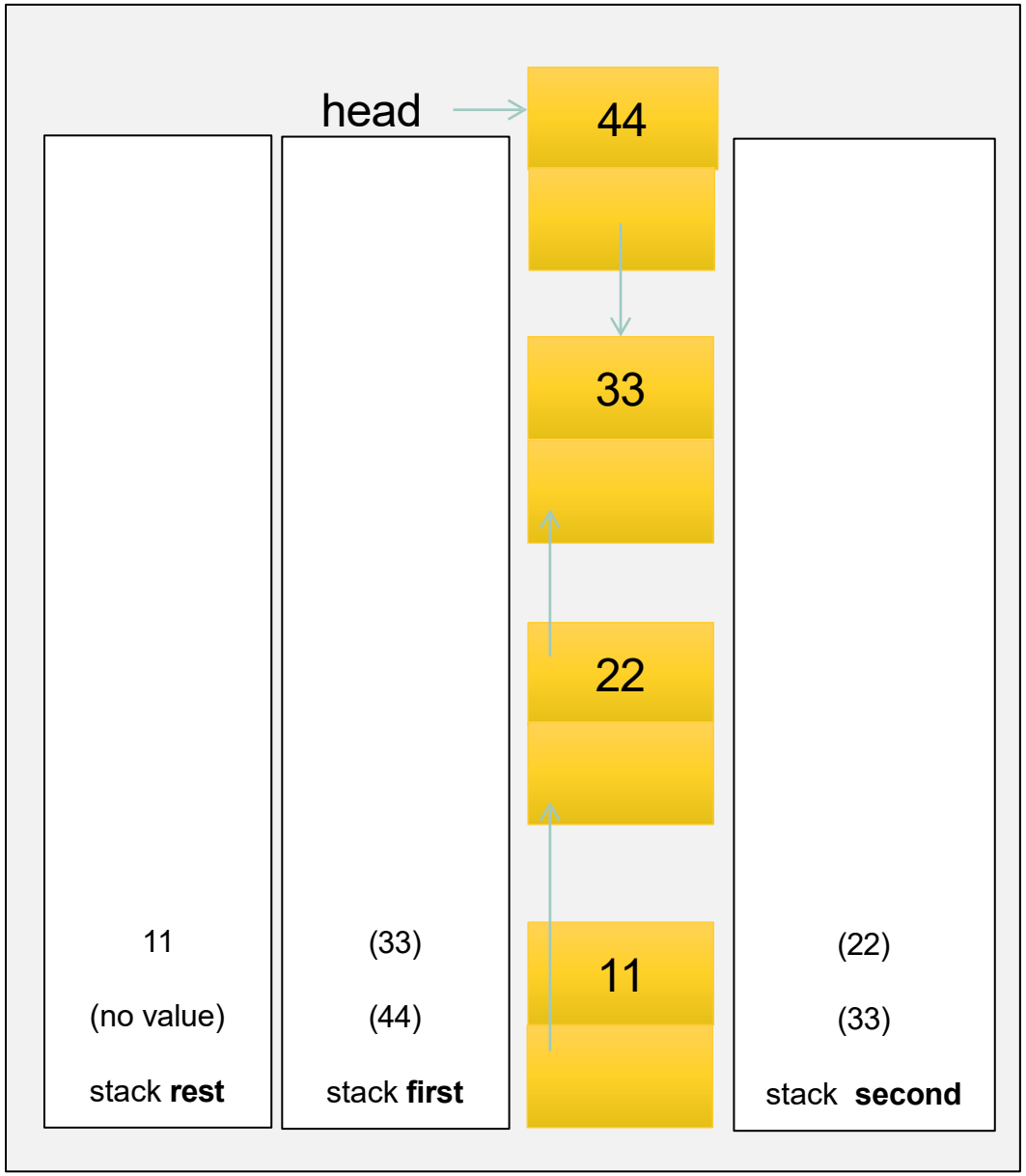
Tracker Line 12+9



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

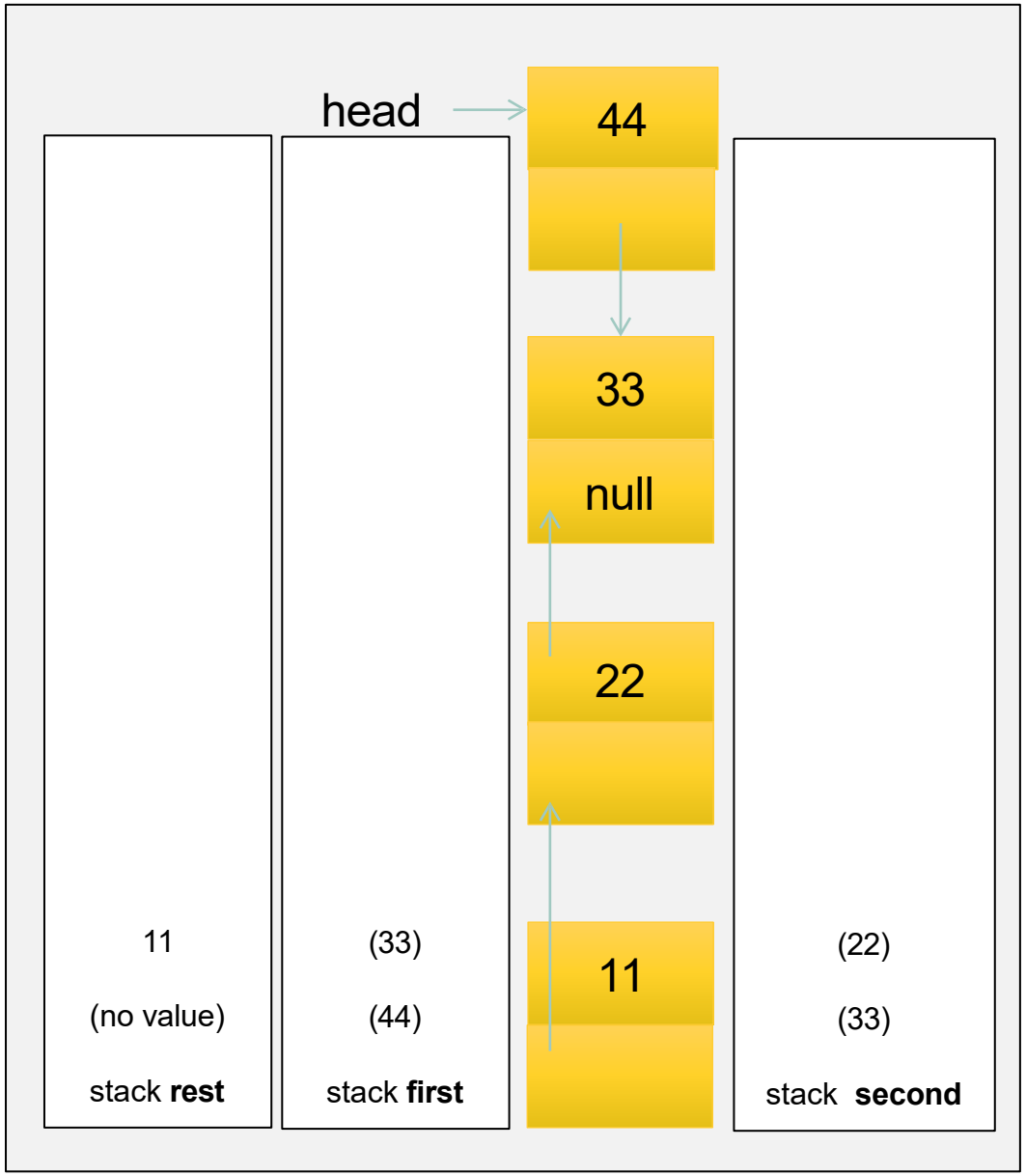
Tracker
Line 10



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

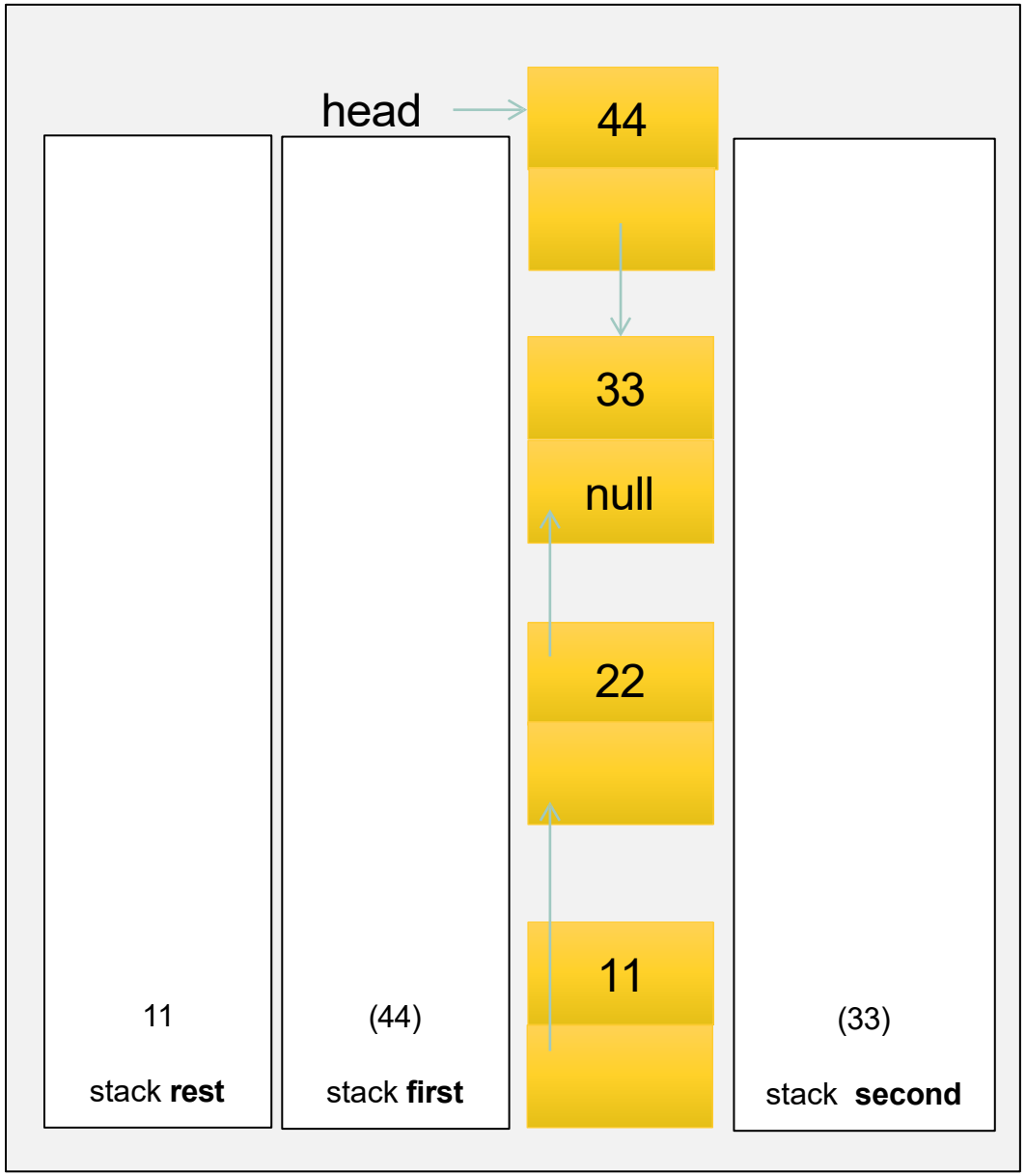
Tracker
Line 11



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

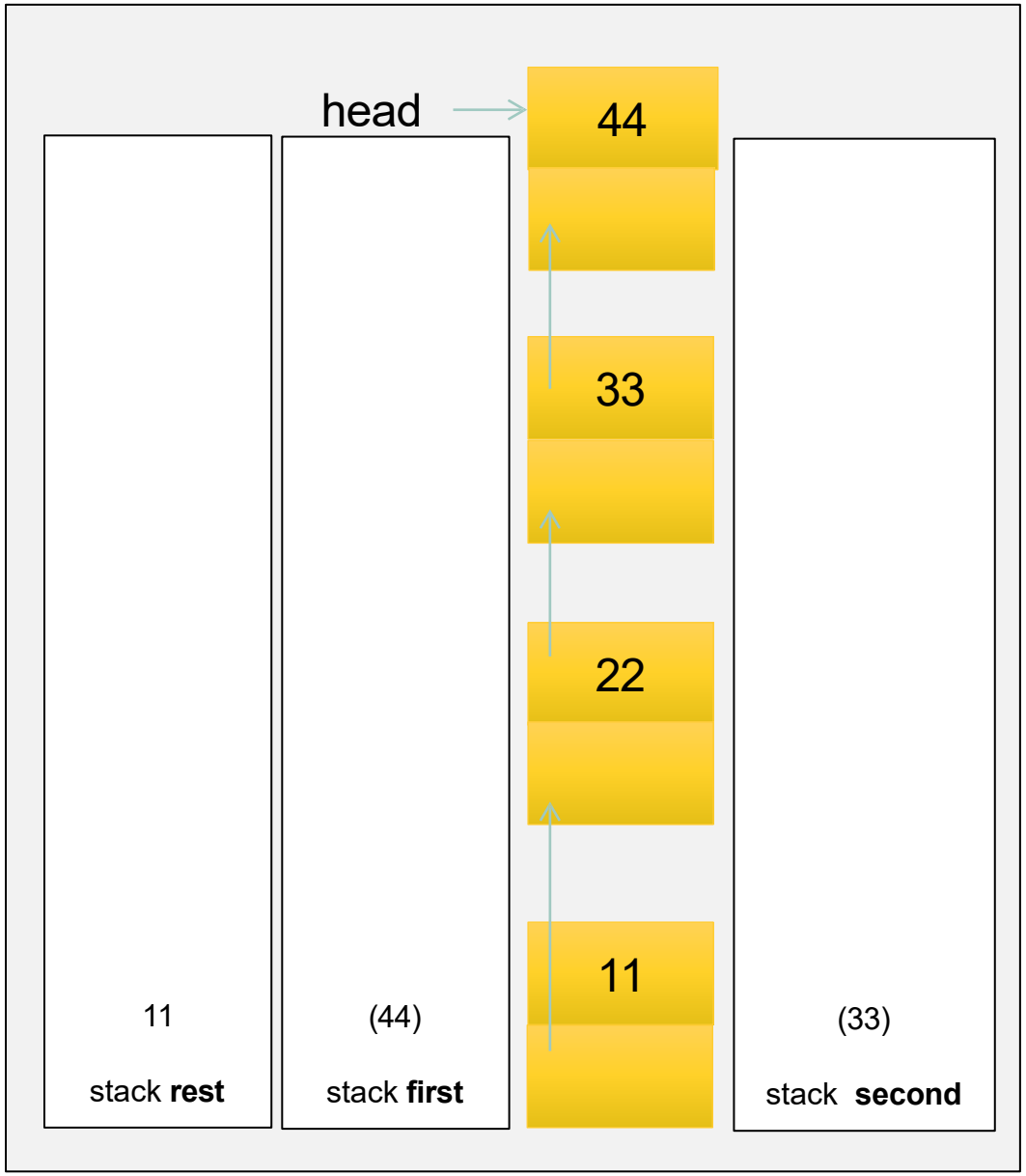
Tracker Line 12+9



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

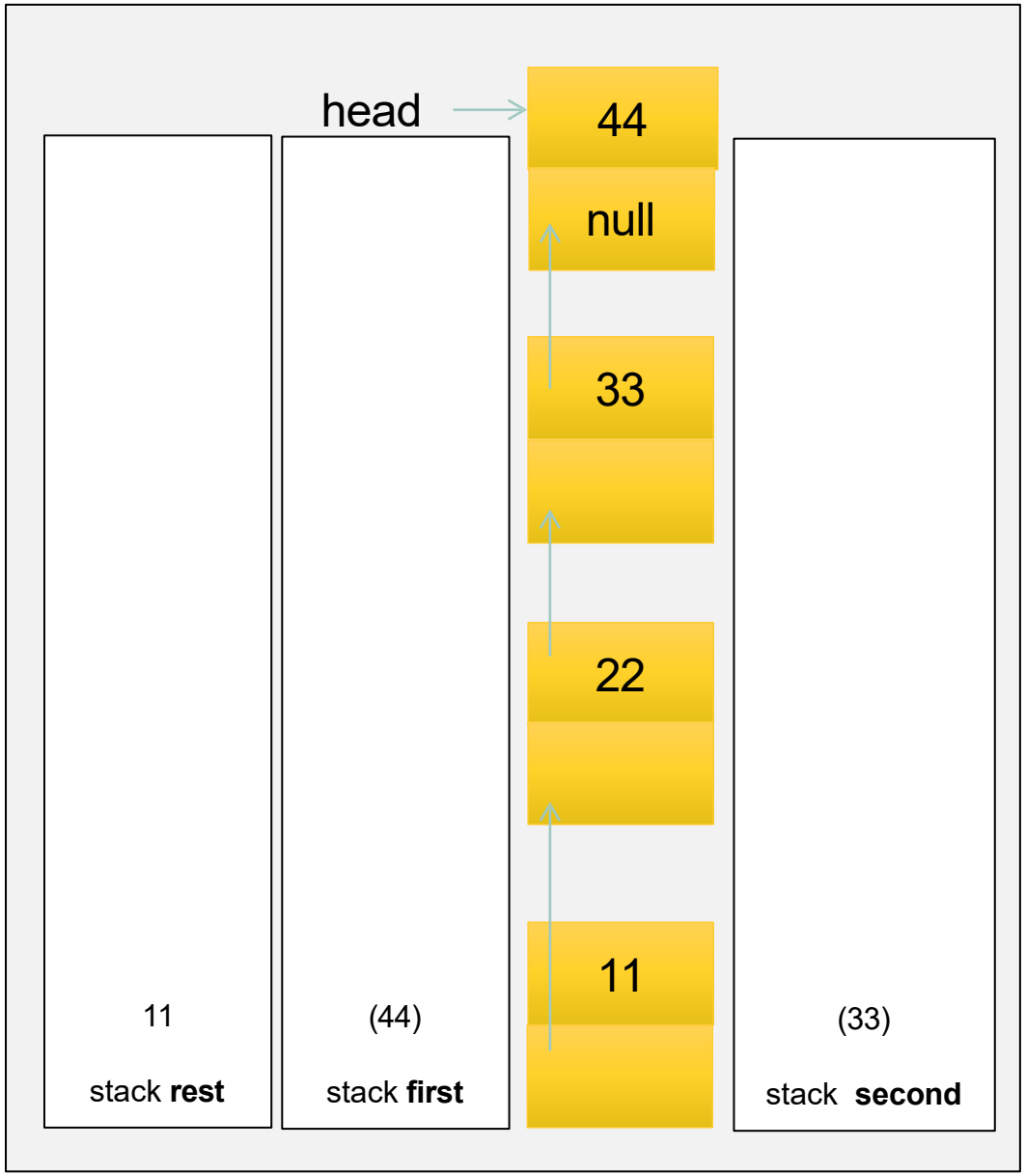
Tracker
Line 10



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

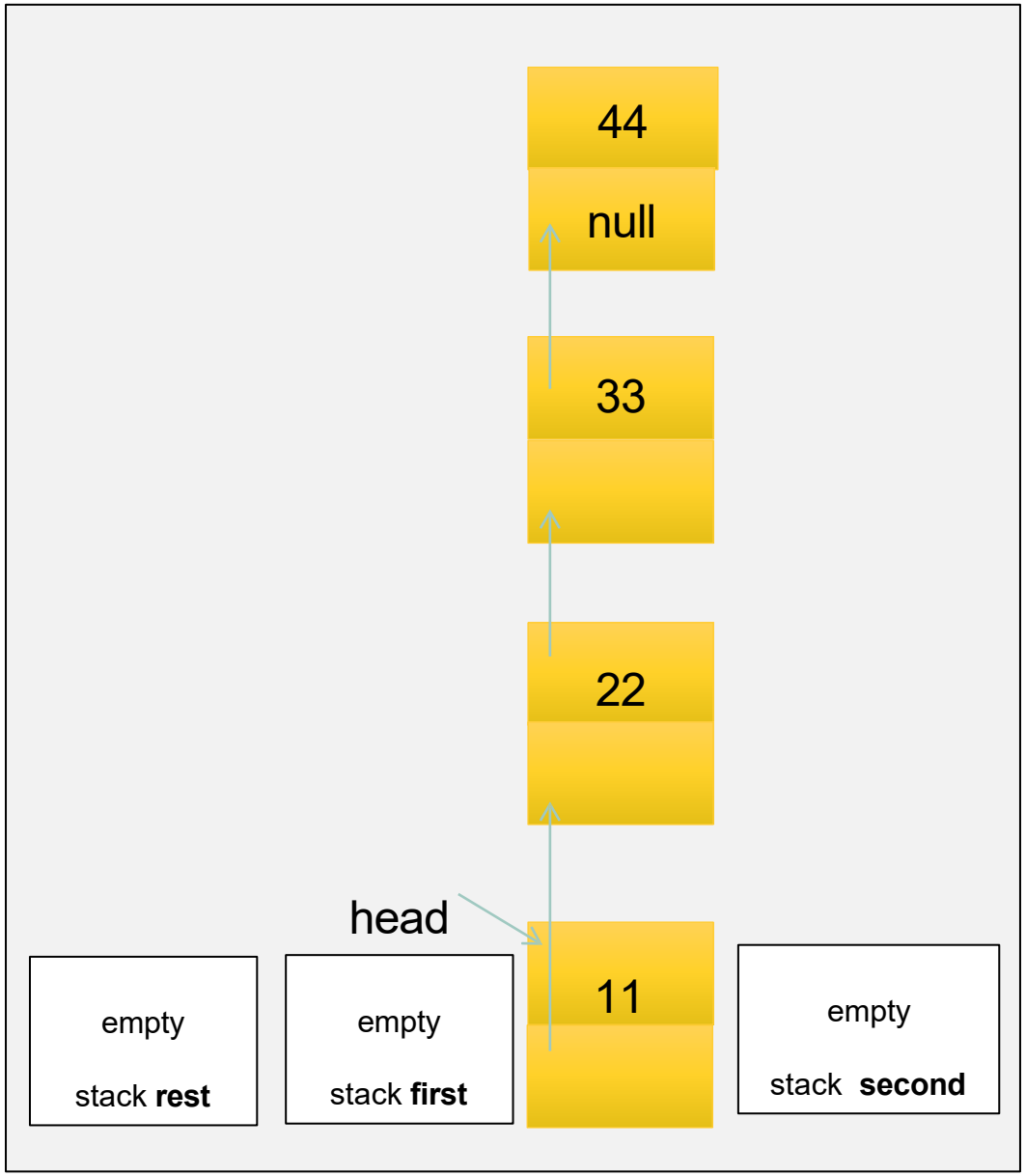
Tracker
Line 11



Recursion Exercise

```java
1  import java.util.*;
2  // For Class Demo only ...
3  public class recursion2 {
4      static Node what(Node first) {
5          if (first == null || first.next == null) {
6              return first;
7          }
8          Node second = first.next;
9          Node rest = what(second);
10         second.next = first;
11         first.next  = null;
12         return rest;
13     }
14
15     public static void main(String[] args)  {
16         LinkedList llist = new LinkedList();
17
18         llist.insertNode(11);
19         llist.insertNode(22);
20         llist.insertNode(33);
21         llist.insertNode(44);
22
23         printLList(llist.head, " ");
24
25         System.out.println();
26
27         llist.head = what(llist.head);
28         printLList(new_llist, " ");
29     }
30 }
```

Tracker
Line 27



What is wrong with this example ?

```java
public class Example {
    public class Recursion_ex2 {

        public static void decrementByTwo(int x) {
            decrementByTwo(x-2);
            System.out.println(x);
        }

        public static void main(String[] args) {
            decrementByTwo(10);
        }
    }
}
```

java.lang.StackOverflowError

Why ?

In Class Exercise

Fix the previous example with your idea !!!